

Automate the Machine, Keep the Voice — \$39

90+

PIECES PER MONTH

3

TWEETS PER SESSION

6

PLATFORMS COVERED

0

VAS NEEDED

Overview

90 pieces of content per month from one person. Not AI slop. Not templated garbage. Actual content with voice, with data, with takes.

The machine handles the volume. You handle the voice.

What you get: The 3-tweet rule with exact prompt templates. A 4-stage content pipeline from raw material to published post. An anti-slop filter that catches AI tells before they go live. A Twitter algorithm breakdown for 2026. A repurposing matrix that turns 1 finding into 5 pieces. A scheduling system that runs on \$0/month.

System Stats

Metric	Value
Content pieces per month	90+ (from 30 min/day human input)
Platforms covered	6 (Twitter/X, LinkedIn, Newsletter, YouTube Shorts, Reddit, community)
VAs required	0

Human time per day	~2 hours (30 min input, 90 min editing and engagement)
Tools required	Claude Code, Python, a Twitter developer account
Monthly cost	Claude Code subscription + \$0 scheduler (local cron)

The Core Insight

Every piece of work you do already generates content. You build something: process tweet. You research a topic: data tweet. You launch a product: proof tweet. You fail publicly: honest take that gets 10x the engagement of anything polished.

The problem is not "not enough to say." The problem is conversion. Raw experience sits in your head and never makes it to a draft. This system handles the conversion automatically. You do the work. The machine writes the first draft from what you did. You edit for 5 minutes. It posts.

The no-blank-page principle: You never start from nothing. The prompt always starts with what you actually did. "You just [did this]. Write 3 tweets about it." The blank page problem is a prompt design problem, not a creativity problem.

What This Is Not

Not a "post more" guide

Posting more bad content faster produces more bad content. This is about converting real work into real signal. Volume is a byproduct of the system, not the goal.

QUALITY FIRST

Not a generic AI writing tool breakdown

This is a specific system with specific prompts, a specific pipeline, and a specific filter stack. The prompts are copy-paste ready. The pipeline is described step by step. The filter is a checklist you run before posting.

SPECIFIC

Not theory

This system runs 90+ pieces of content per month across a real account. The 3-tweet rule is a standing rule in the codebase. The anti-slop filter is a real Claude prompt that runs before every post. The scheduling queue is a real CSV file with a real Python cron job reading it.

BATTLE-TESTED

The 3-Tweet Rule

Every build session, research sprint, product launch, or client meeting generates 3 tweets and 1 thread minimum. Non-negotiable. This is Rule 9 in the operating system.

The rule in plain language: You do not finish a work session without 3 drafted tweets in the queue. Not ideas. Not "I should post about this." Drafted, filtered, queued.

The Exact Prompt Template

You just [DESCRIBE WHAT YOU DID]. Write 3 tweets about this for a solopreneur audience.
Rules: No AI vocabulary. No em dashes. No "game-changing." No "dive into."
Each tweet: specific, consequence-first, no more than 3 sentences.
One tweet per line. No preamble.

Why consequence-first? Twitter readers decide in 0.3 seconds whether to keep reading. The consequence of what you learned or built is more

interesting than the explanation of it. Start with the result, then explain how.

Example: Prompt to Output

Input to Claude:

You just ran 14,000 cold emails over 6 weeks and tracked open rates, reply rates, and meeting booked rates by subject line format, send time, and lead source. Write 3 tweets about this for a solopreneur audience.

Rules: No AI vocabulary. No em dashes. No "game-changing." No "dive into."

Each tweet: specific, consequence-first, no more than 3 sentences.

One tweet per line. No preamble.

Output (3 tweets, ready to queue):

14,000 cold emails, 6 weeks. The subject line "quick question" outperformed every other format by 31%. Nobody opens "following up on my last email."

Tuesday 8 AM beats Friday 11 AM by 2.4x on reply rate. Not because people are more receptive. Because your email isn't competing with 40 others from the weekend.

Inbound leads from content replied at 6.8%. Cold scrape leads replied at 1.2%. The gap is trust. Build the audience first, cold email second.

The Thread Formula

For longer content (research, multi-step processes, results), use the thread structure:

Tweet	Purpose	Formula
-------	---------	---------

1 (Hook)	Stop the scroll	[Specific number] [specific claim]. [Proof or tension].
2-9 (Body)	Deliver the value	One point per tweet. Number, then insight. Never two ideas in one tweet.
10 (CTA)	Convert the reader	Single action. Follow for more on [topic]. Or link to product.

Time math: 20 minutes of writing (describing what you did) generates the full prompt input. Claude generates 10 tweet drafts in 15 seconds. You edit for 5 minutes. 25 minutes total, 10 pieces of content, one thread that could get 200K impressions if the hook lands.

Content Types by Work Type

What You Did	Content Type	Format
Built something new	Process tweet	"Here's how I built [X] in [time]. Step by step:"
Found data or research	Data tweet	"[Stat] from [source]. Here's what it actually means:"
Launched a product	Proof tweet	"Day 1 results: [numbers]. What worked, what didn't:"
Failed at something	Honest take	"I [did X]. It didn't work. Here's exactly why:"
Had a client meeting	Insight tweet	"Client said [thing]. That's actually the most common [problem]."
Read something interesting	Curation tweet	"Most people missed this [study/finding/quote]. Here's why it matters:"

Where Drafts Live

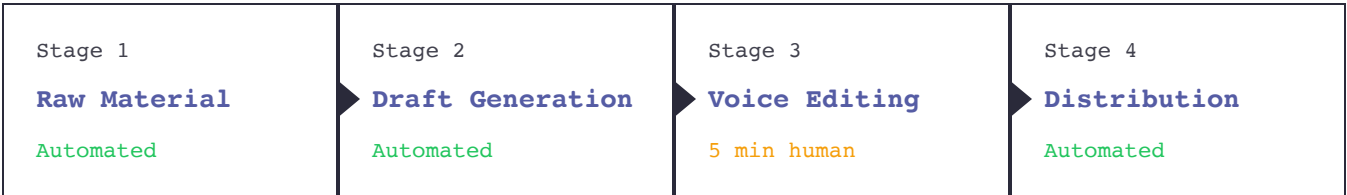
All drafted tweets go to one file: `CONTENT/social/posting_queue/queue.csv` .
Format:

```
platform,content,post_time,status
twitter,"14,000 cold emails, 6 weeks...",2026-03-25 08:00,PENDING
twitter,"Tuesday 8 AM beats Friday...",2026-03-26 08:00,PENDING
```

The scheduler reads this file and posts automatically. You draft once. It posts on schedule. See the Scheduling section for the full cron setup.

Content Pipeline

Four stages from raw material to published post. Two are automated. One is 5 minutes of human time. One is automated again.



Stage 1: Raw Material Collection

Sources that feed the pipeline automatically:

Build sessions

Every coding session, design sprint, or product build. What you made, how long it took, what broke, what worked. Captured via session notes or git commit messages.

AUTO-SCRAPED

Reddit and Twitter scraping

Scrapers run every 4-6 hours. They pull high-signal posts from your niche: what people are asking, complaining about, and paying for. Raw findings go

into a staging file.

AUTO-SCRAPED

Competitor monitoring

Top accounts in your niche, tracked weekly. What's getting traction. What angles nobody has covered yet. Which hooks are working right now.

AUTO-SCRAPED

Alpha research findings

Any research you do: pricing analysis, user interviews, product comparisons, test results. Numbers and observations get flagged as content seeds automatically.

HUMAN INPUT

Stage 2: Draft Generation

The engagement bait converter prompt. Takes any raw finding and returns 3+ formatted posts:

Raw finding: [PASTE THE FINDING OR OBSERVATION]

Convert this into 3 Twitter posts for a [NICHE] audience.

Rules:

- No AI vocabulary (no "delve," "dive into," "seamlessly," "leverage," "game-changi
- No em dashes
- Consequence first, explanation second
- Specific numbers over vague claims
- Max 3 sentences per tweet
- No hashtags
- No preamble, just the tweets

Format: one tweet per line, blank line between each.

The engagement bait classifier: Before generating content from any finding, run it through this 5-point test. Specific tool or platform named? Verifiable numbers? Executable today with info given? Plausible at stated scale? Real method or recycled wisdom? If 3+ are yes: full content piece. If 1-2: extract the specific kernel first, then write from that.

Stage 3: Voice Editing

This is the only stage that requires you. It takes 5 minutes per batch of 10 drafts. You are looking for:

AI tells (em dashes, banned words, passive voice) that the anti-slop filter missed

Claims without numbers you can add

Hedging you can remove ("it might be worth considering" becomes "do this")

The first word of the tweet. If it's "I," rewrite the opening.

Anything that sounds like it was written by a tool, not a person with skin in the game

See the Anti-Slop Filter section for the full checklist.

Stage 4: Distribution

The queue script. Reads the CSV, finds the oldest PENDING post whose scheduled time has passed, posts it via API, marks it POSTED:

```
# posting_queue_runner.py (simplified)
import csv
import tweepy
from datetime import datetime

def post_pending():
    rows = []
    with open('CONTENT/social/posting_queue/queue.csv', 'r') as f:
        reader = csv.DictReader(f)
        rows = list(reader)

    for row in rows:
```

```

    if row['status'] != 'PENDING':
        continue
    post_time = datetime.fromisoformat(row['post_time'])
    if post_time > datetime.now():
        continue
    # post to Twitter API v2
    client = tweepy.Client(bearer_token=BEARER, ...)
    try:
        client.create_tweet(text=row['content'])
        row['status'] = 'POSTED'
    except Exception as e:
        row['status'] = 'FAILED'
    break # one post per cron run

# write updated statuses back
with open('CONTENT/social/posting_queue/queue.csv', 'w') as f:
    writer = csv.DictWriter(f, fieldnames=rows[0].keys())
    writer.writeheader()
    writer.writerows(rows)

```

Time Investment

Activity	Time	Who
Describe what you did (prompt input)	15–20 min/day	Human
Voice editing of drafts	5–10 min/batch	Human
Engagement (replies to your posts)	20–30 min/day	Human
Reply farming (10 replies to large accounts)	15–20 min/day	Human
Scraping, draft generation, scheduling, posting	0 min	Automated

Total human time: 55–80 minutes per day produces 3+ posts + engagement activity. The rest runs without you.

Anti-Slop Filter

AI-generated content has fingerprints. Readers who use AI themselves spot them in 2 seconds. The anti-slop filter removes those fingerprints before anything goes live.

The 6 Words That Flag AI Content Instantly

Word / Phrase	Why It Signals AI	Replace With
dive into	Overused Claude/GPT opener	Just start with the thing. "Here's how:"
delve	Nobody says this in speech. Ever.	Look at, examine, check
game-changing	Vague superlative with no data	Specific improvement with numbers
leverage (as a verb)	Corporate consultant speak, AI default	Use, run, apply
seamlessly	Means nothing. Nothing is seamless.	Delete it. If true, show it with a number.
cutting-edge	Every AI calls its output cutting-edge	State what's actually new or different

The No Em Dash Rule

Em dashes (like this one) are the single most reliable AI signal in 2026. Claude and GPT use them at roughly 3-4x the rate of human writers. Any reader who has used AI in the last 12 months will clock them immediately. The rule is simple: never use em dashes in content. Use a period. Or a comma. Or end the sentence and start a new one.

Sentence Structures That Kill Engagement

Passive voice

Before: "Content was generated by the system and distributed across platforms." After: "The system generates and distributes content automatically." Active is shorter, stronger, and sounds like a human wrote it.

FIX: ACTIVE VOICE

Hedged claims

Before: "It might be worth considering whether this approach could potentially improve your results." After: "This will improve your reply rate. Run it for 14 days and measure." Hedging signals either the writer doesn't believe what they're saying, or they're an AI trying to be safe.

FIX: COMMIT

Jargon stacking

Before: "Leverage this comprehensive framework to seamlessly optimize your holistic content strategy." This sentence means nothing. After: "Post 3 times per day. Reply to 10 large accounts. Do this for 90 days." Numbers and actions beat noun phrases every time.

FIX: SPECIFICS

The 5-Question Voice Check

Run every draft through these before queuing:

1. Would a real person say this out loud without cringing?
2. Is there a specific number I could add to make this more concrete?
3. Is there hedging I can remove (might, could, potentially, perhaps)?
4. Does it lead with the consequence, not the explanation?
5. Is there any word I would never use in a text message to a friend?

If any question surfaces a problem, fix it before moving on. The voice check takes 60 seconds per tweet when you are fast at it.

Before and After

BEFORE (AI SLOP)

This comprehensive guide dives into the leveraging of AI tools to seamlessly optimize your content creation workflow – helping you produce game-changing output at scale.

AFTER (HUMAN VOICE)

This shows you the exact system I use to produce 90 content pieces per month without a VA. 30 minutes of input per day. The rest is automated.

Voice Model Injection

The most reliable way to make Claude write like you: give it 5 examples of content you've already written that you're proud of. Then prepend this to every content prompt:

Write in the voice of these examples. Same directness. Same sentence length. Same specificity level. Same willingness to state things without hedging.

Example 1: [your tweet]

Example 2: [your tweet]

Example 3: [your tweet]

Example 4: [your tweet]

Example 5: [your tweet]

Now write [the new content].

The voice model is a living document. Keep a file called `CONTENT/voice_model.txt` with your 10–20 best pieces. Update it monthly. Add new examples when you write something that performs above average. Remove old examples that no longer represent how you write. Inject from this file automatically in the draft generation prompt.

Twitter Algorithm 2026

What it rewards, what it punishes, and the non-obvious tactics that compound over 90 days.

What the Algorithm Rewards

Signal	Why It Matters	How to Optimize
Dwell time	Reading = signal of quality. Scrolling past = no signal.	First line must create a question the reader needs answered. Never front-load the answer.
Saves / Bookmarks	Highest-quality signal. Means the reader values it enough to return.	Reference content (lists, frameworks, resources) saves at 3-5x the rate of takes.
Replies from large accounts	When a 50K account replies, their audience sees your post.	Reply to large accounts first. Make them want to respond. Ask specific questions.
Engagement velocity (first 30 min)	The algorithm decides how far to push the tweet based on early performance.	Post when you can be online for 30 minutes after. Reply to every comment in that window.

What It Punishes

External links in the tweet body

Twitter's algorithm suppresses tweets with outbound links. The standard workaround: post the link in a reply to your own tweet. Or post link-free content and put the URL in the first comment immediately after posting.

CRITICAL

Low-ratio accounts

If you follow 2,000 accounts and have 400 followers, your reach is suppressed. The ratio signals low-value account to the algorithm. Fix: unfollow aggressively, or build followers faster through the reply strategy below.

WATCH

Posting at wrong times

Dead-hour posts get no engagement in the first 30 minutes. No early engagement means no algorithmic push. The post is effectively invisible. See the timing table below.

WATCH

Best Posting Times

Platform	Best Time	Why
Twitter/X	7-9 AM EST	Morning scroll before work, high intent, less noise than midday
Twitter/X	12-1 PM EST	Lunch scroll, high engagement window for threads
Twitter/X	6-8 PM EST	Evening wind-down, high personal finance and productivity reach
LinkedIn	Tue-Thu 8-10 AM	B2B audience, midweek professional mindset
Newsletter	Tuesday 7 AM	Not Monday (inbox overwhelm), not Friday (pre-weekend check-out)
Reddit	9 AM-12 PM EST	US morning traffic, best upvote velocity

The Reply Strategy

10 quality replies per day to large accounts in your niche grows your account faster than 3 original tweets. This is not opinion. It is how the algorithm works. Your reply is shown to the large account's followers. If the reply is good, they check your profile.

The compound effect: 10 replies/day for 90 days = 900 high-visibility touchpoints in your niche. Each one is a tiny ad that costs \$0 and targets exactly the audience you want. The accounts you reply to are already doing the targeting work for you.

What a good reply looks like:

Adds information the original post didn't include

Disagrees with a specific point (politely) using data

Shares a directly relevant result from your own experience

Asks a specific question that invites the original poster to engage back

What a bad reply looks like: "Great post!" "This." "Agreed." "Love this perspective." These replies hurt you because they signal low-value account.

The Hook Formula

The thread hook format that consistently outperforms all others:

[Specific number] [Specific claim]. [Proof or tension].

Working examples

"I ran 14,000 cold emails. Here's what worked:"

"Day 44. \$0 revenue. Here's what I'm doing differently:"

"3 tools replaced my \$2,400/month VA stack:"

"I analyzed 200 viral threads from 2025. They all share 1 structure:"

COPY THESE

The Controversy Play

The highest-engagement format: a claim most people disagree with, backed by specific numbers, with an invitation for pushback. The algorithm rewards replies. Controversy generates replies. The math is simple.

Use carefully: The controversy must be genuine. You must actually believe it and have data for it. Manufactured controversy where you don't believe the take damages trust faster than it builds reach. The goal is "provocative but defensible," not "wrong for clicks."

Repurposing Matrix

One research finding becomes 5 content pieces across 5 platforms. Each is a different angle, not a copy. The same finding told 5 ways to 5 different audiences.

The 1-to-5 Matrix

Format	Platform	Angle	Length
Twitter thread	Twitter/X	"Here's what I learned" (numbered, hook-driven)	10 tweets
LinkedIn post	LinkedIn	"Here's the business implication" (measured, professional)	300-500 words
Newsletter section	Email	"Here's the full context" (nuanced, reader assumes more time)	300-400 words
Short video script	YouTube Shorts / TikTok	"Here's the visual proof" (fast, hook in second 1)	60 seconds

Community post	Reddit / Discord / Slack	"What do you think?" (question format, invite discussion)	100-150 words
-------------------	-----------------------------	--	------------------

The 5 Prompts

Twitter Thread Prompt

Turn this finding into a 10-tweet thread. Hook tweet first. 8 numbered points
CTA tweet last. Each tweet: one idea, specific number if possible, max 3 sent
No em dashes. No AI vocabulary. No "leverage" or "seamlessly."
Finding: [PASTE FINDING]

LinkedIn Post Prompt

Write a LinkedIn post about this finding. Audience: B2B professionals and fou
Tone: direct, data-backed, slightly more measured than Twitter. 300-400 words
Focus on the business implication, not just the tactic.
No bullet points. Prose paragraphs. End with 1 question to the reader.
Finding: [PASTE FINDING]

Newsletter Section Prompt

Write a newsletter section about this finding. 300-400 words.
The reader has 3 minutes. Give them the full context: what you found,
why it matters, and what they should do about it. More nuanced than Twitter.
Readers expect the backstory. Include it.
Finding: [PASTE FINDING]

Short Video Script Prompt

Write a 60-second video script about this finding. Hook in the first 3 seconds (a specific claim or visual cue). Body: 3 points, 15 seconds each. CTA: 5 seconds. Format: [HOOK] [POINT 1] [POINT 2] [POINT 3] [CTA] Write it as spoken words, not prose. Short sentences. Natural speech rhythm. Finding: [PASTE FINDING]

Community Post Prompt

Write a community post about this finding for a [NICHE] forum or Discord. 100-150 words. Open with the finding directly. No "I wanted to share" preamble. End with a genuine question that invites people to share their own experiences. No self-promotion. Pure value + question. Finding: [PASTE FINDING]

The Weekly Repurposing Schedule

Day	Action	Source
Monday	Original research or build session. Write Twitter thread.	Your work
Tuesday	LinkedIn repurpose of Monday's finding	Monday thread
Wednesday	Newsletter section (expand Monday's thread with more context)	Monday thread
Thursday	Short video script from Monday's hook tweet	Monday thread hook
Friday	Community post (question version of Monday's finding)	Monday finding

Time math: 1 Monday research session (2 hours). 4 days of 10-minute repurposing (prompt, quick edit, queue). 1 week of content across 5 platforms from one source.

The angles must differ. The Twitter thread is "here's what I learned." The LinkedIn post is "here's what this means for your business." The newsletter is "here's the full story with context." The video is "watch me show you." The community post is "what's your take?" Same finding. Five different conversations. None of them feels like a copy to a reader who sees all five.

Scheduling System

The queue format, the free tools, the CSV import method, and the local Python scheduler that runs for \$0/month.

The Posting Queue Format

A single CSV file that every part of the system reads and writes:

```
platform,content,post_time,status
twitter,"14,000 cold emails over 6 weeks. The subject line...",2026-03-25 08:00,PENDING
linkedin,"The business implication of running 14,000 cold...",2026-03-26 09:00,PENDING
twitter,"Tuesday 8 AM beats Friday 11 AM by 2.4x on reply...",2026-03-27 08:00,PENDING
newsletter,"This week I ran 14,000 cold emails and tracked...",2026-03-27 07:00,PENDING
```

Free Scheduling Options

Tool	Free Tier	Best For	Catch
Buffer	10 scheduled posts	Multi-platform, easy UI	10 post limit per account
Publer	Limited posts	Visual calendar view	Caps kick in at volume
TweetDeck	Free (X subscription required)	Twitter only, powerful	Requires X Blue
Local Python cron	\$0 forever	High volume, full control	Your machine must be on

Recommendation: For low volume (under 30 posts/month), Buffer free tier is enough. For 90+ pieces/month, the local Python cron is the only option that doesn't hit caps or cost money. The script is 50 lines and runs on a Mac with launchd or Linux with crontab.

The CSV Import Method (30-Day Content Batch)

Write 30 days of content in one 2-3 hour session. Save as CSV. Import to Buffer or Hootsuite. Done for the month. The prompt:

```
I need 30 Twitter posts for the month of [MONTH] on the topic of [NICHE].
Rules:
- No em dashes. No AI vocabulary (delve, dive into, seamlessly, leverage, game-chan
- Each tweet: consequence first, specific numbers, max 3 sentences.
- Mix: 10 process tweets (how I do X), 10 data tweets (here's what I found),
    5 honest takes (what didn't work), 5 controversial claims (backed by data).
- Output as CSV: date,platform,content,status
    Start dates from [START DATE], one tweet per day at 8 AM EST.

Voice examples:
[PASTE 5 EXAMPLES OF YOUR BEST TWEETS]
```

The Local API Poster

Posts directly via Twitter API v2. No third-party tool. No monthly cost. Runs on cron every hour, checks the queue, posts anything due:

```
# crontab entry (runs every hour)
0 * * * * /usr/bin/python3 /path/to/posting_queue_runner.py >> /path/to/logs/poster
```

```
# posting_queue_runner.py (full version)
import csv, os, sys
from datetime import datetime
import tweepy

QUEUE_PATH = 'CONTENT/social/posting_queue/queue.csv'
LOG_PATH   = 'CONTENT/social/posting_queue/post_log.txt'

def get_client():
```

```

return tweepy.Client(
    consumer_key=os.environ['TWITTER_API_KEY'],
    consumer_secret=os.environ['TWITTER_API_SECRET'],
    access_token=os.environ['TWITTER_ACCESS_TOKEN'],
    access_token_secret=os.environ['TWITTER_ACCESS_SECRET']
)

def run():
    with open(Queue_PATH, 'r') as f:
        rows = list(csv.DictReader(f))

    now = datetime.now()
    posted = False

    for row in rows:
        if row['status'] != 'PENDING':
            continue
        if row['platform'] != 'twitter':
            continue
        scheduled = datetime.fromisoformat(row['post_time'])
        if scheduled > now:
            continue

        try:
            client = get_client()
            client.create_tweet(text=row['content'])
            row['status'] = 'POSTED'
            with open(LOG_PATH, 'a') as log:
                log.write(f"{now.isoformat()} POSTED: {row['content'][:60]}\n")
            posted = True
        except Exception as e:
            row['status'] = 'FAILED'
            with open(LOG_PATH, 'a') as log:
                log.write(f"{now.isoformat()} FAILED: {e}\n")
        break # one post per cron run

    if posted or any(r['status'] in ('POSTED', 'FAILED') for r in rows):
        with open(Queue_PATH, 'w', newline='') as f:
            writer = csv.DictWriter(f, fieldnames=rows[0].keys())
            writer.writeheader()
            writer.writerows(rows)

if __name__ == '__main__':
    run()

```

What to Manually Approve Before Scheduling

Posts about specific people

Any tweet that names a specific person, company, or account. The 5-minute human review is worth it to avoid a public mistake that's permanent on the internet.

MANUAL REVIEW

Posts with statistics you haven't verified

If Claude generated a number and you didn't check the source, don't schedule it. Specific wrong numbers get ratio'd. Vague correct numbers don't. Check every stat before it goes live.

MANUAL REVIEW

Strong takes on active controversies

Anything that references a live news event, public dispute, or rapidly evolving situation. Your opinion from Tuesday can look badly-informed by Thursday. If it has a short shelf life, post it manually when you're online to handle the response.

MANUAL REVIEW

The final system in one paragraph: You do work. You describe it in a prompt. Claude drafts 3 tweets and a thread in 15 seconds. You spend 5 minutes editing for voice using the anti-slop filter. The posts go into a CSV. A Python cron job reads the CSV every hour and posts whatever is due. You check the queue once per week to add more content. The engagement (replies, reply farming) you handle live, 30 minutes per day. That is the entire system. 90+ pieces per month. \$0/month beyond Claude Code.

